

| Introduction to Python

Python is one of the most popular and widely used programming languages in the world. It was created to make programming simpler, more readable, and easier to learn compared to many older programming languages.

Python is considered a *high-level, interpreted, and general-purpose programming language*. This means it can be used in many different fields such as web development, automation, cybersecurity, data analysis, game development, scientific computing, and especially Artificial Intelligence.

Main Features of Python

1. Simple and Readable Syntax
2. Large Ecosystem of Libraries
3. Cross-Platform Compatibility
4. Strong Community Support
5. Fast Development Speed

Dynamic Typing in Python

One of the most important features of Python is that it is a dynamically typed language. This means the programmer does not need to manually specify the data type of a variable when creating it.

```
In [1]: x = 10      # Integer
        name = "Ali" # String
        price = 9.99 # Float
```

Variable Types Can Change Dynamically

Python even allows changing the type of a variable during program execution:

```
In [2]: data = 10
        data = "Hello"
```

The type() function

In Python, you can check the type of any data that you are using. To check the data type, you can use the type() function.

```
In [3]: type(10)
```

```
Out[3]: int
```

```
In [18... type(2.718)
```

```
Out[18... float
```

```
In [19... type("hello")
```

```
Out[19... str
```

| Strings

```
In [1]: x = "Python is great!"
        print(x)
        print(type(x))

        y = 'Python is great!'
        print(y)
        print(type(y))
```

```
Python is great!
<class 'str'>
Python is great!
<class 'str'>
```

Note: In Python, there is no difference between using single quotes ('...') and double quotes ("...") for defining a string; both create the same str object. The choice is primarily a matter of style or convenience.

Multiline strings

Note: Multi-line strings cannot be enclosed in regular double quotes (""). To define a string that spans multiple lines, you must use triple quotes (""" or ''').

```
In [21]: print("Hello, World!
           It's great to be here!")
```

```
Cell In [21], line 1
      print("Hello, World!
            ^
```

SyntaxError: unterminated string literal (detected at line 1)

```
In [ ]: print("""Hello, World!
           It's great to be here!""")
```

```
Hello, World!
It's great to be here!
```

Access characters and substrings

```
In [29]: my_string = "I love python"

# get character by referring to index
b = my_string[0]
print(b)
# get last element
b = my_string[-1]
print(b)
# Substrings with slicing
b = my_string[2:6] # Note that the last index is not included
print(b)
b = my_string[:5] # from beginning
print(b)
b = my_string[6:] # until the end
print(b)
b = my_string[::2] # start to end with every second item
print(b)
b = my_string[::-1] # reverse the string with a negative step:
print(b)
```

```
I
n
love
I lov
python
Ilv yhn
nohtyp evol I
```

String Concatenation

```
In [24... message_1 = "I'm learning Python"
message_2 = "and it's fun!"
```

```
In [25... message_1 + message_2
```

```
Out[25... "I'm learning Pythonand it's fun!"
```

```
In [26... message_1 + " " + message_2
```

```
Out[26... "I'm learning Python and it's fun!"
```

```
In [27... message = f'{message_1} {message_2}'
message
```

```
Out[27... "I'm learning Python and it's fun!"
```

In Python, the `len()` function is used to calculate the length of a string by counting the number of characters inside it.

```
In [32... print("length of message:" + str(len(message)))
length of message:33
```

```
In [33... print(f"length of message: {len(message)}")
length of message: 33
```

Mixing strings with computations or data: f-Strings

```
In [34... print(f"The temperature 75F in degrees celsius is {(75 - 32) * 5 / 9}C")
The temperature 75F in degrees celsius is 23.88888888888889C
```

f-String Reminder: Expressions inside an f-string must be enclosed in curly braces (`{}`).

If the curly braces are omitted, Python will treat the text as a literal string and print it exactly as written instead of evaluating the mathematical expression.

```
In [2]: print(f"The temperature 75F in degrees celsius is (75 - 32) * 5 / 9 C")
The temperature 75F in degrees celsius is (75 - 32) * 5 / 9 C
```

```
In [38... print(f"Isabel is {28/7:.0f} years old.")
Isabel is 4 years old.
```

In `f"{28/7:.0f}"`, the `:.0f` part tells Python to display the result of `28/7` without any decimal places. You don't need to worry too much about the details, but the `f` in `:.0f` indicates that the number is a floating-point number and should be formatted accordingly. This means it will be rounded to the nearest whole number and displayed without any decimal part.

| Lists

```
In [39... numbers = [1, -1, 10]
print(numbers)
print(f'list has {len(numbers)} elements')

[1, -1, 10]
list has 3 elements
```

```
In [40... # list manipulation
# element replacement
numbers = [1, 2, 3, 10, 15, 2, -10]
numbers[0] = 0
numbers[2:4] = [-10, -10]
print(numbers)

[0, 2, -10, -10, 15, 2, -10]
```

Adding elements to a list

```
In [41... # append: Adds a new element at the end of a list
countries = ['United States', 'India', 'China', 'Brazil']
countries.append('Canada')
countries
```

```
Out[41... ['United States', 'India', 'China', 'Brazil', 'Canada']
```

```
In [42... # insert: Adds a new element at a specific position
countries = ['United States', 'India', 'China', 'Brazil']
countries.insert(0, 'Canada')
countries
```

```
Out[42... ['Canada', 'United States', 'India', 'China', 'Brazil']
```

```
In [43... # joining lists using the + operators
countries_2 = ['UK', 'Germany', 'Austria']
countries + countries_2
```

```
Out[43... ['Canada',
'United States',
'India',
'China',
'Brazil',
'UK',
'Germany',
'Austria']
```

Remove an element

```
In [44... countries = ['United States', 'India',
               'China', 'Brazil']
print(countries)

# .remove() Removes the first matching value
countries.remove('United States')
print(countries)

['United States', 'India', 'China', 'Brazil']
['India', 'China', 'Brazil']
```

```
In [4]: countries = ['United States', 'India', 'China', 'Brazil']

print(countries)

# .pop() Removes and returns the last item
removed_country = countries.pop()

print(removed_country)
print(countries)
```

```
['United States', 'India', 'China', 'Brazil']
Brazil
['United States', 'India', 'China']
```

```
In [ ]: countries = ['United States', 'India', 'China', 'Brazil']

print(countries)

# .pop(index) Removes and returns the item at the specified index
removed_country = countries.pop(1)

print(removed_country)
print(countries)
```

```
['China', 'Brazil']
```

```
In [ ]: countries = ['United States', 'India', 'China', 'Brazil']

print(countries)

# del Removes an item by index
del countries[2]

print(countries)
```

```
['Brazil']
```

```
In [5]: countries = ['United States', 'India', 'China', 'Brazil']

print(countries)

# del can remove multiple items using slicing
del countries[1:3]

print(countries)
```

```
['United States', 'India', 'China', 'Brazil']
['United States', 'Brazil']
```

```
In [6]: countries = ['United States', 'India', 'China', 'Brazil']

print(countries)

# del can delete the entire list
del countries

print(countries) # NameError: countries is not defined
```

```
['United States', 'India', 'China', 'Brazil']
```

```
-----
NameError                                Traceback (most recent call last)
Cell In [6], line 8
      5 # del can delete the entire list
      6 del countries
----> 8 print(countries)

NameError: name 'countries' is not defined
```

```
In [7]: countries = ['United States', 'India', 'China', 'Brazil']

print(countries)

# .clear() Removes all items from the list
countries.clear()

print(countries)

['United States', 'India', 'China', 'Brazil']
[]
```

Summary

`.remove(value)` : Removes the first matching value

`.pop()` : Removes and returns the last item

`.pop(index)` : Removes and returns the item at the specified index

`del list[index]` : Removes an item by index

`del list[start:end]` : Removes multiple items

`.clear()` : Removes all items but keeps the list variable

`del list_name` : Deletes the entire list variable

| If statements

```
In [47... # if statements
my_list = [1, 2, -4, 10]
number_to_find = 10

if number_to_find in my_list:
    # call METHOD "index" of the list OBJECT:
    print(f'{number_to_find} is the element no {my_list.index(number_to_find)}')
else:
    print(f'{number_to_find} is not found in list')

10 is the element no 3
```

```
In [8]: temperature = 29
if temperature < 10:
    print("cold")
elif temperature < 25:
    print("normal")
elif temperature < 35:
    print('hot')
else:
    print("very hot")
```

hot

Comparing data in Python

```
In [9]: is_tommy_my_friend = True
is_isabel_my_friend = False
is_isabel_older_than_me = False
```

```
In [10]: print(is_tommy_my_friend)
```

True

```
In [61]: type(is_isabel_older_than_me)
```

Out[61]: bool

```
In [62]: isabel_age = 28
daniel_age = 30
tommy_age = 30
```

```
In [63]: print(isabel_age > daniel_age)
```

False

```
In [64]: is_isabel_older_than_daniel = isabel_age > daniel_age
print(is_isabel_older_than_daniel)
```

False

```
In [65]: print(isabel_age <= daniel_age)
```

True

```
In [66]: print(tommy_age == daniel_age)
```

True

```
In [67]: print(tommy_age != daniel_age)
```

False

Logical Operators

```
In [68]: print(is_tommy_my_friend and is_isabel_my_friend)
```

False

```
In [69]: print(is_tommy_my_friend or is_isabel_my_friend)
```

True

```
In [70... print(not False)
```

```
True
```

Loops

The for loop is a frequently used loop in Python. It allows us to loop through an iterable object (e.g., list, dictionaries) and perform the same action to each element.

```
In [3]: countries = ['United States', 'India',  
                    'China', 'Brazil']  
for i in range(len(countries)):  
    print(countries[i])
```

```
United States  
India  
China  
Brazil
```

```
In [15... countries = ['United States', 'India',  
                  'China', 'Brazil']  
for country in countries:  
    print(country)
```

```
United States  
India  
China  
Brazil
```

```
In [72... for country in countries:  
        if country == "United States":  
            print(country)
```

```
United States
```

```
In [18... for i, country in enumerate(countries):  
        print("{i}: {country}".format(i=i, country=country))
```

```
0: United States  
1: India  
2: China  
3: Brazil
```

List comprehension

A elegant and fast way to create a new list from an existing list.

List comprehension consists of an expression followed by a for statement inside square brackets.

```
In [74... a = [1, 2, 3, 4, 5, 6, 7, 8]  
b = [i **3 for i in a]  
print(b)
```

```
[1, 8, 27, 64, 125, 216, 343, 512]
```

```
In [1]: data = ['peter', 'Paul', None, 'MARY', 'gUIDO']  
[s.capitalize() for s in data if s is not None]
```

```
Out[1]: ['Peter', 'Paul', 'Mary', 'Guido']
```

```
In [19... data = [1, 2, 3, 4, 5, 6, 7, 8]
labels = ['even' if i % 2 == 0 else 'odd' for i in data]
```

```
In [20... labels
```

```
Out[20... ['odd', 'even', 'odd', 'even', 'odd', 'even', 'odd', 'even']
```

list comprehensions are faster than for loops

```
In [ ]: import time
n = 1_000_000

t1 = time.time()
my_list = [i**2 for i in range(0, n)]
t2 = time.time()

my_list = []
for i in range(n):
    my_list.append(i**2)
t3 = time.time()
t_loop = t3 - t2
t_com = t2 - t1
print(f"list comprehensions are {100*(t_loop-t_com)/t_loop:.1f} % faster")
```

list comprehensions are 17.6 % faster

Eng. Najwa Alghamdi